

PRODUCT DOCUMENTATION

Statistical Data Analysis

User guide & technical documentation — descriptive statistics, t-tests, ANOVA, chi-square, correlation, regression, multiple regression, PCA and factor analysis, computed entirely in the browser.

Creator	**Dr. Asif Ahmad**
Product	Statistical Data Analysis — version 1.0
Category	Productivity
Published by	Asif SaaS Studio
Date	20 October 2026
Format	Single-file web application (index.html, 71.1 KB)

Private by design: the analysed data never leaves the user's device. Every statistic in this product is verified against scipy, the scientific reference implementation (Section 13).

Contents

1. What this product does
2. Getting started in three steps
3. Preparing your data
4. Descriptive statistics
5. T-tests
6. ANOVA (one-way)
7. Chi-square test of independence
8. Correlation
9. Regression
10. Multiple regression
11. Principal Component Analysis (PCA)
12. Factor analysis
13. How correctness was verified
14. Reading p-values and effect sizes
15. Errors and what they mean
16. Privacy and security model
17. Limitations and honest scope
18. Frequently asked questions
19. Product information and credits
20. Appendix — the tested statistics core (verbatim)

1. What this product does

Statistical Data Analysis turns raw numbers into defensible conclusions. You provide data — a class's test scores, a survey, experimental measurements, business figures — and the app summarises it, tests hypotheses about it, measures relationships between variables, builds predictive models, and reduces many variables to the few underlying dimensions that matter. Every analysis returns three things: a **statistical table** in the standard academic layout, a **chart** where a picture helps, and a **plain-language interpretation** that states the conclusion, its strength, and its caution.

Analysis	The question it answers
Descriptive statistics	What does this variable look like — centre, spread, shape?
One-sample t-test	Does this mean differ from a known or target value?
Independent t-test	Do two groups differ on a measurement?
Paired t-test	Did the same subjects change (before vs after)?
One-way ANOVA	Do three or more groups differ?
Chi-square	Are two categorical variables associated?
Correlation	How strongly do two numeric variables move together?
Regression	How does Y change with X — and can we predict Y?
Multiple regression	What does each predictor contribute, holding the others constant?
PCA	Which few components capture most of the variation?
Factor analysis	Which latent factors explain the correlations among variables?

The product is built for researchers, teachers presenting analysis to students, and professionals who need trustworthy statistics without installing specialist software — and who cannot send their data to a server.

2. Getting started in three steps

Step 1 — Load data. Drag a file onto the upload area (or tap to browse), paste data into the text box, or press Load sample dataset to explore with 24 students' study data. The app confirms the load: rows, columns, each column's type (numeric or categorical) and value count, plus a preview of the first rows.

Step 2 — Choose an analysis. Pick one of the nine tabs. Each tab shows exactly the selectors that analysis needs — variable drop-downs or tick-boxes — populated from your columns.

Step 3 — Run and read. Press Run analysis. Read the table for the numbers, the chart for the shape, and the green interpretation box for the conclusion. Press Download report (HTML) to save a formatted report file — tables, charts and interpretation included — for your records, your students, or your manuscript's methods file.

> **Tip.** Run Descriptive statistics first on any new dataset. Most analysis mistakes are visible there: wrong column types, outliers, near-constant variables, and unexpectedly small samples.

3. Preparing your data

3.1 Supported formats

Format	Notes
CSV (.csv)	The standard. Any delimiter — comma, semicolon, tab or pipe — is detected automatically.
TSV / TXT (.tsv, .txt)	Tab- or otherwise-delimited plain text, handled like CSV.
JSON (.json)	Either an array of objects (`[{"Score":72,"Group":"A"}, …]`) or an array of rows with a header row first.
Paste	Copy from a spreadsheet and paste — pasted tables arrive tab-delimited and are detected.
Excel (.xlsx)	**Not read directly.** In Excel: File → Save As → CSV UTF-8, then upload the CSV. A spreadsheet file is a zipped package of XML; a faithful reader is a large library that cannot fit this product's single-file size budget — the app says so honestly rather than pretending.

3.2 Rules the parser follows

- **First row = column names.** Blank names become Var1, Var2…; duplicate names are made unique.
- **Quoted fields** may contain delimiters and line breaks; doubled quotes (`""`) become one quote, per the CSV standard.
- **Numbers** may use thousands separators (`1,000.5`) or a decimal comma (`1,5`) — both are recognised. Scientific notation works too.
- **Missing values:** empty cells and the tokens `NA`, `N/A`, `null`, `NaN`, `-`, `.` are treated as missing, never as zero.
- **Column types** are inferred: a column whose non-missing values are all numbers is numeric; anything else is categorical. A single stray label (e.g. "72 points") makes the whole column categorical — clean it in your file if a variable is mistyped.
- **File limit:** 5 MB (roughly 50,000 rows of typical data).

3.3 How missing data is handled per analysis

Univariate and two-variable analyses use every row where the needed values exist (pairwise). Multiple regression, PCA and factor analysis use complete cases only (rows with no missing value among the selected variables) and tell you how many rows qualified.

4. Descriptive statistics

Use it to understand each numeric variable before any testing. **Select** one or more numeric variables and run.

Statistic	Meaning
Mean / Median / Mode	Average / middle value / most frequent value. Mean ≈ median suggests a symmetric distribution.
SD, Variance	Typical distance of values from the mean (SD is in the variable's own units).
Min, Q1, Q3, Max, IQR	The range and the middle 50% of values (IQR = Q3 − Q1) — robust to outliers.
Skewness	Asymmetry. Between −0.5 and 0.5 ≈ symmetric; beyond ±1 is strongly skewed (long tail in the sign's direction).
Kurtosis (excess)	Tail heaviness vs the normal curve; 0 = normal-like.
95% CI of the mean	The range in which the population mean plausibly lies, given this sample.

The histogram shows the shape directly: one hump or two, symmetry, and stragglers. **Reading the interpretation:** it compares mean and median through the skewness and tells you which centre to report for a skewed variable (the median).

5. T-tests

5.1 One-sample t-test

Question: does the mean of one variable differ from a test value μ_0 (a standard, a target, last year's mean)? **Select** the variable, type μ_0, run. The table gives t, df, the two-sided p, and a confidence interval for the difference. The interpretation states the verdict in words.

5.2 Independent two-sample t-test

Question: do two groups differ? **Select** the numeric variable and a grouping column with exactly two categories. Choose **Welch** (default, recommended — it does not assume equal variances) or **pooled** (classic Student test). Alongside t and p you get **Cohen's d**, the standardised effect size: ≈0.2 small, 0.5 medium, 0.8 large. The box plot shows the two distributions side by side.

5.3 Paired t-test

Question: did the same units change? **Select** the two measurements (e.g. PreTest and PostTest); rows are paired by position. The test works on the within-pair differences — the histogram shows those differences — and reports Cohen's d for the change.

> **Assumptions, honestly.** T-tests assume roughly normal data (or a large enough sample for the central limit theorem) and independent observations. With very small or heavily skewed samples, treat borderline p-values with caution — and always report the effect size beside the p-value.

6. ANOVA (one-way)

Question: do three or more groups differ on a numeric variable? (With two groups, use a t-test.) **Select** the numeric variable and the grouping column. The output has two tables: the group summaries (n, mean, SD) and the ANOVA table — between/within sums of squares, the F statistic, p, and **η² (eta-squared)**: the share of the variable's variation associated with group membership (≈0.01 small, 0.06 medium, 0.14 large).

Reading the interpretation: it names the highest and lowest groups and the variance share. A significant F says somewhere among the groups there is a difference — follow up with pairwise t-tests, using a stricter threshold (e.g. Bonferroni: divide 0.05 by the number of comparisons) to protect against false positives. A built-in post-hoc procedure is a documented limitation (Section 16).

7. Chi-square test of independence

Question: are two categorical variables associated — do the row proportions change across columns? **Select** the two categorical variables. The app builds the contingency table of counts, computes χ^2, df, p, and **Cramér's V** (0–1; ≈0.1 small, 0.3 medium, 0.5 large association).

> **Expected-count caution.** The χ^2 p-value is trustworthy when expected counts are adequate. If more than 20% of cells have expected counts below 5, the interpretation warns you — combine rare categories in your data and re-run.

8. Correlation

Question: how do numeric variables move together? **Choose** Pearson (linear relationship; the default) or Spearman (rank-based; safer for skewed data, outliers, or ordinal scores), **tick** two or more variables, and run. The matrix shows every pairwise r with significance stars (* $p < .05$, ** $p < .01$, *** $p < .001$), and a scatter plot with the fitted line appears for the first two variables.

|r| range	Strength	Plain meaning
0.00 – 0.10	Negligible	Essentially no linear association.
0.10 – 0.30	Small	A real but weak tendency.
0.30 – 0.50	Moderate	A noticeable relationship.
0.50 – 0.70	Strong	Clearly patterned together.
0.70 – 1.00	Very strong	Near-locked movement (check for redundancy).

The sign gives direction: positive — both rise together; negative — one rises as the other falls. The interpretation names the strongest pair in your data. **Correlation is not causation** — the app repeats this in the output because it matters: a third variable, or reverse causality, may explain the association.

9. Regression

Question: how does an outcome Y change with a predictor X — and how well can X predict Y? **Select** Y and X and run. The fitted line appears on the scatter plot, and the app prints the equation: Y = intercept + slope × X.

Output	How to read it
B (slope)	Expected change in Y for a 1-unit increase in X.
SE, t, p	Precision of B; whether the slope differs significantly from zero.
R^2	Share of Y's variation the line explains (0–1; 0.25 ≈ a quarter).
Adjusted R^2	R^2 corrected for model size — the honest comparison number.
Residual SE	Typical prediction error, in Y's units.

> **Worked reading.** "For each additional study hour, PostTest rises by 3.4 marks on average ($p < .001$); study time explains 41% of the variation in scores." Slope for the story, p for the confidence, R^2 for the usefulness — report all three.

10. Multiple regression

Question: with several predictors at once, which ones genuinely matter? **Select** the outcome Y and tick the predictors. Each coefficient is now a unique contribution — the effect of that predictor **holding the others constant** — which is why a variable significant alone can fade in company, and vice versa.

- **B** — change in Y per unit of the predictor, others fixed (in original units).

- **β (standardised)** — the same effect in standard-deviation units. Compare |β| across predictors to rank their importance fairly.
- **Model F and p** — whether the predictors jointly explain Y better than the mean alone.

> **Collinearity.** If two predictors are near-duplicates of each other, the model cannot separate their effects and the design matrix becomes singular. The app detects this and refuses with a clear message naming the problem — remove the redundant predictor and re-run. (Milder collinearity inflates standard errors; watch for large SEs with strong individual correlations.)

Data needs: complete cases must outnumber predictors comfortably — a working rule is at least 10–15 observations per predictor for stable estimates.

11. Principal Component Analysis (PCA)

Question: can many correlated variables be summarised by a few components that keep most of the information? **Tick** three or more numeric variables (two works, but summarises little). The app standardises the variables (correlation-based PCA), so units never distort the result.

Output	How to read it
Eigenvalue	Variance carried by the component. Kaiser's rule: keep components > 1.
% / cumulative %	Share of total variation; researchers often keep enough components to reach ≈70–80%.
Loadings	Correlation of each variable with each component. Large loadings (|·| ≳ 0.5) define what a component "is".
Scree plot	Bars of % variance per component; keep the components before the "elbow" where bars flatten.

The interpretation applies the Kaiser rule for you and states the combined variance the kept components explain. Name components by their high-loading variables (e.g. a component loading on all test scores might be "overall achievement").

12. Factor analysis

Question: what latent factors explain why these variables correlate? Where PCA summarises data, factor analysis models it: each variable = a mix of common factors plus its own uniqueness. **Tick** three or more variables, choose the number of factors (start with 2), and run. The method is **principal axis factoring** with iterated communalities, followed

by **varimax rotation** — the classical combination taught in research-methods courses — implemented from the statistical definitions.

Output	How to read it
Rotated loadings	Variable–factor correlations after rotation. |loading| ≳ 0.4 usually counts as belonging to the factor.
Communality h^2	Share of the variable's variance the factors explain. Low h^2 (< ≈0.3) = the variable doesn't fit the factor story.
SS loadings / % variance	Each factor's explanatory weight, in variable units and percent.

The interpretation names each factor by its two strongest variables — your job is to give the factor a meaningful label. Practical guidance: factor analysis wants data — aim for at least 5–10 observations per variable and variables that genuinely intercorrelate; with near-zero correlations there is nothing for factors to explain.

13. How correctness was verified

A statistics product is only worth using if its arithmetic is provably right. The engine was verified three independent ways during development, and the results below are part of the product record.

13.1 Against the scientific reference (scipy)

41 of 41 numeric checks passed. Every statistic the app computes was compared with scipy, the reference scientific library: descriptive statistics including adjusted skewness and kurtosis; the t, F, chi-square and normal distribution functions themselves; one-sample, pooled, Welch (including its fractional degrees of freedom) and paired t-tests; ANOVA's F and p; chi-square; Pearson and Spearman coefficients and p-values; simple regression (slope, intercept, p, R^2); multiple regression coefficients and R^2 against a least-squares solution; and PCA eigenvalues against a matrix-eigensolver reference.

13.2 Structural recovery test

Synthetic data was generated from a known **two-factor** structure (three variables driven by factor 1, three by factor 2). The app's factor analysis recovered the structure exactly — variables clustered onto the correct factors, every primary loading above 0.6.

13.3 A bug the testing caught

Verification also proved its worth: an early version of the t critical-value routine bisected the wrong tail of the distribution, which would have made every "95%" confidence interval actually a 97.5% interval. The comparison tests caught it; the function was corrected and now matches the reference critical values to three decimals (e.g. 2.306 at df = 8, 2.086 at df = 20). Parser and validation behaviour passed a further 20 checks: quoted CSV fields,

escaped quotes, delimiter detection, thousands separators, decimal commas, both JSON shapes, missing-value tokens, and the rejection of constant columns, collinear predictors, and undersized samples.

14. Reading p-values and effect sizes

- **p-value:** the probability of data at least this extreme if the null hypothesis were true. $p < .05$ is the conventional "significant" line — a convention, not a law of nature. The app reports exact p-values (e.g. $p = .013$; $p < .001$ below that) so you can judge.
- **Stars:** * $p < .05$ · ** $p < .01$ · *** $p < .001$.
- **Effect size** answers "how much?", which p cannot: Cohen's d (differences), η^2 (ANOVA), Cramér's V (association), r (correlation), R^2 (models). A tiny effect can be significant in a huge sample; a large effect can miss significance in a small one. Report both — the app always pairs them.
- **Confidence intervals** show the plausible range for the true value; an interval that excludes the null value (0 difference, μ_0) agrees with a significant test.

15. Errors and what they mean

Message (short form)	Cause and remedy
File larger than 5 MB	Trim rows/columns or split the file; re-export as CSV.
Needs a header row and at least one data row	The first row must be column names, with data beneath.
Grouping must have exactly 2 categories	Independent t-test only. For 3+ groups use ANOVA; check for typos creating phantom categories.
Each group needs at least 2 values	A group has too little data after missing values are excluded.
Predictors are collinear	Two predictors duplicate each other; remove one.
Not enough complete cases	Missing values shrank the usable rows; add data or drop sparse variables.
Variable is constant	A selected variable never varies — it cannot correlate or load; remove it.
Column appears as text	A numeric column typed categorical usually has a label mixed in ("72 points"); clean the source file.

16. Limitations and honest scope

- **Excel files** are converted via Save-As-CSV (Section 3.1) — there is no hidden partial support.
- **ANOVA** reports the omnibus test; dedicated post-hoc procedures (Tukey HSD) are not built in — the interpretation tells you how to follow up correctly.
- **Two-group categorical predictors in regression** are not dummy-coded automatically; enter numeric codings (0/1) as a numeric column if you need them.
- **Robust/non-parametric alternatives** (Mann–Whitney, Kruskal–Wallis) beyond Spearman are not included in v1.0.
- **Very large data:** the 5 MB cap and in-browser computation suit teaching and research datasets, not millions of rows.
- Results are standard textbook statistics; for publication, report the effect sizes and intervals the app provides alongside p-values, as Section 14 advises.

17. Privacy and security model

- **Everything is local.** Parsing and all statistics run in the page, on the device. Data is never uploaded — there is no server to upload to.
- **Nothing is kept.** The app uses no localStorage, cookies, accounts or analytics. Reloading the page forgets the dataset completely.
- **No third parties.** No external scripts, fonts, or network calls of any kind — the page works fully offline.
- **Sandbox-safe.** No service workers, no permission prompts, no parent-window access, so it runs inside the studio's embedded frame exactly as specified.

18. Frequently asked questions

My numbers loaded as "categorical" — why? One non-numeric cell (a label, a % sign, a footnote) types the whole column as text. Clean the source column so every value is a plain number, then reload.

Welch or pooled t-test? Welch, unless you have a specific reason to assume equal variances — Welch stays valid when variances differ and loses almost nothing when they don't.

Pearson or Spearman? Pearson for roughly linear, well-behaved data; Spearman for ranks, ordinal scores, skew, or outliers — it measures monotone association on ranks.

PCA or factor analysis? PCA to compress variables into components; factor analysis to explain correlations with latent factors. If you can't say which story you want, start with PCA's variance table.

How many factors/components should I keep? Components: Kaiser (> 1) plus the scree elbow plus cumulative ≈70–80%. Factors: start with theory, check that each factor has at least 2–3 strong loadings and a nameable meaning.

Can I cite this tool in a paper? Cite the method (e.g. "Welch's independent-samples t-test") as usual; the tool is the calculator. Its verification against scipy is documented in Section 13.

19. Product information and credits

Field	Detail
Product	Statistical Data Analysis
Version	1.0 (20 October 2026)
Creator	**Dr. Asif Ahmad**
Publisher	Asif SaaS Studio
Category / slug	Productivity / statistical-data-analysis
Deliverable	index.html — 71.1 KB, single file, all CSS and JavaScript inline
Requirements	Any modern browser; no server, no installation, no account
Verification	41/41 statistical checks vs scipy; factor-structure recovery test; 20 parser/validation checks (Section 13)

Appendix — the tested statistics core (verbatim)

The code below is the complete statistical engine of the product, reproduced exactly as it ships inside `index.html` (interface and chart code omitted). This is the module that passed every verification in Section 13: parsing, descriptive statistics, the distribution functions (incomplete beta and gamma), correlations, t-tests, ANOVA, chi-square, least-squares regression, the Jacobi eigensolver, PCA, varimax rotation and principal-axis factor analysis.

```
var MISSING_TOKENS = {"":1, "na":1, "n/a":1, "null":1, "nan":1, "-":1, ".":1, "missing":1};

/* ---------- Delimited-text parsing (CSV/TSV/TXT) ---------- */
function sniffDelimiter(text){
  var firstLine = text.split(/\r\n|\r|\n/)[0] || "";
  var cands = [",", ";", "\t", "|"], best = ",", bestN = -1, inQ = false, counts = {",":0,";":0,"\t":0,"|":0};
  for (var i = 0; i < firstLine.length; i++){
    var ch = firstLine[i];
    if (ch === '"') inQ = !inQ;
    else if (!inQ && counts.hasOwnProperty(ch)) counts[ch]++;
  }
  cands.forEach(function(c){ if (counts[c] > bestN){ bestN = counts[c]; best = c; } });
  return best;
}
function parseDelimited(text, delim){
  var rows = [], row = [], field = "", inQ = false, i, ch;
```

```
  text = String(text).replace(/^\uFEFF/, "");
  function endField(){ row.push(field); field = ""; }
  function endRow(){ endField(); rows.push(row); row = []; }
  for (i = 0; i < text.length; i++){
    ch = text[i];
    if (inQ){
      if (ch === '"'){ if (text[i + 1] === '"'){ field += '"'; i++; } else inQ = false; }
      else field += ch;
    } else if (ch === '"') inQ = true;
    else if (ch === delim) endField();
    else if (ch === "\n") endRow();
    else if (ch === "\r"){ if (text[i + 1] === "\n") i++; endRow(); }
    else field += ch;
  }
  if (field.length || row.length) endRow();
  return rows.filter(function(r){ return !(r.length === 1 && r[0].trim() === ""); });
}
/* JSON: array of objects, or array of arrays (first row = headers) */
function parseJsonData(text){
  var data = JSON.parse(text);
  if (!Array.isArray(data) || !data.length) throw new Error("JSON must be a non-empty array.");
  if (Array.isArray(data[0])){
    return data.map(function(r){ return r.map(function(v){ return v === null || v === undefined ? "" :
String(v); }); });
  }
  if (typeof data[0] === "object"){
    var keys = [];
    data.forEach(function(o){ Object.keys(o).forEach(function(k){ if (keys.indexOf(k) < 0) keys.push(k); }); });
    var out = [keys];
    data.forEach(function(o){ out.push(keys.map(function(k){ var v = o[k]; return v === null || v ===
undefined ? "" : String(v); })); });
    return out;
  }
  throw new Error("JSON must be an array of objects or an array of rows.");
}
/* Build the dataset from a rows grid (first row = header) */
function buildDataset(grid){
  if (!grid || grid.length < 2) return { error: "The data needs a header row and at least one data row." };
  var headers = grid[0].map(function(h, i){ h = String(h).trim(); return h || ("Var" + (i + 1)); });
  var seen = {};
  headers = headers.map(function(h){ if (seen[h]){ seen[h]++; return h + "_" + seen[h]; } seen[h] = 1; return
h; });
  var ncol = headers.length, body = grid.slice(1);
  var cols = headers.map(function(name, ci){
    var raw = body.map(function(r){ return ci < r.length ? String(r[ci]).trim() : ""; });
    var numeric = true, vals = [];
    raw.forEach(function(v){
      if (MISSING_TOKENS[v.toLowerCase()]){ vals.push(null); return; }
      var numStr = v;
      if (/^-?\d{1,3}(,\d{3})+(\.\d+)?$/.test(v)) numStr = v.replace(/,/g, "");       // 1,000.5 thousands
      else if (/^-?\d+,\d+$/.test(v)) numStr = v.replace(",", ".");                  // 1,5 decimal comma
      var n = Number(numStr);
      if (isFinite(n)) vals.push(n); else { numeric = false; vals.push(v); }
    });
    if (!numeric){
      vals = raw.map(function(v){ return MISSING_TOKENS[v.toLowerCase()] ? null : v; });
      // a mostly-numeric column with a few labels stays categorical — but count numerics
      return { name: name, numeric: false, values: vals };
    }
    return { name: name, numeric: true, values: vals };
  });
  return { names: headers, n: body.length, cols: cols };
}
function colNumbers(ds, idx){ return ds.cols[idx].values.filter(function(v){ return v !== null; }); }
function paired(ds, i, j){
  var x = [], y = [], a = ds.cols[i].values, b = ds.cols[j].values;
  for (var k = 0; k < ds.n; k++) if (a[k] !== null && b[k] !== null){ x.push(a[k]); y.push(b[k]); }
  return { x: x, y: y };
}

/* ---------- Descriptive statistics ---------- */
function mean(a){ var s = 0; a.forEach(function(v){ s += v; }); return s / a.length; }
function variance(a, sample){
  var m = mean(a), s = 0; a.forEach(function(v){ s += (v - m) * (v - m); });
  return s / (a.length - (sample ? 1 : 0));
}
function quantile(a, q){
```

```
  var s = a.slice().sort(function(x, y){ return x - y; });
  var pos = (s.length - 1) * q, base = Math.floor(pos), rest = pos - base;
  return s[base + 1] !== undefined ? s[base] + rest * (s[base + 1] - s[base]) : s[base];
}
function describe(a){
  var n = a.length, m = mean(a), sd = Math.sqrt(variance(a, true));
  var m2 = 0, m3 = 0, m4 = 0;
  a.forEach(function(v){ var d = v - m; m2 += d * d; m3 += d * d * d; m4 += d * d * d * d; });
  m2 /= n; m3 /= n; m4 /= n;
  var skew = (n > 2 && m2 > 0) ? (Math.sqrt(n * (n - 1)) / (n - 2)) * (m3 / Math.pow(m2, 1.5)) : 0;
  var kurt = (n > 3 && m2 > 0) ? ((n - 1) / ((n - 2) * (n - 3))) * ((n + 1) * (m4 / (m2 * m2) - 3) + 6) : 0;
  var freq = {}, mode = null, modeN = 1;
  a.forEach(function(v){ freq[v] = (freq[v] || 0) + 1; if (freq[v] > modeN){ modeN = freq[v]; mode = v; } });
  var sem = sd / Math.sqrt(n);
  var tc = tCrit(0.975, n - 1);
  var sorted = a.slice().sort(function(x, y){ return x - y; });
  return { n: n, mean: m, median: quantile(a, 0.5), mode: mode, sd: sd, variance: variance(a, true),
    min: sorted[0], max: sorted[n - 1], q1: quantile(a, 0.25), q3: quantile(a, 0.75),
    iqr: quantile(a, 0.75) - quantile(a, 0.25), sum: m * n, sem: sem,
    ciLo: m - tc * sem, ciHi: m + tc * sem, skew: skew, kurtosis: kurt, sorted: sorted };
}

/* ---------- Special functions & distributions ---------- */
function lgamma(x){
  var c = [0.99999999999980993, 676.5203681218851, -1259.1392167224028, 771.32342877765313,
    -176.61502916214059, 12.507343278686905, -0.13857109526572012, 9.9843695780195716e-6,
1.5056327351493116e-7];
  if (x < 0.5) return Math.log(Math.PI / Math.sin(Math.PI * x)) - lgamma(1 - x);
  x -= 1; var a = c[0], t = x + 7.5;
  for (var i = 1; i < c.length; i++) a += c[i] / (x + i);
  return 0.5 * Math.log(2 * Math.PI) + (x + 0.5) * Math.log(t) - t + Math.log(a);
}
function betacf(a, b, x){
  var MAXIT = 200, EPS = 3e-12, FPMIN = 1e-300;
  var qab = a + b, qap = a + 1, qam = a - 1, c = 1, d = 1 - qab * x / qap;
  if (Math.abs(d) < FPMIN) d = FPMIN; d = 1 / d; var h = d;
  for (var m = 1; m <= MAXIT; m++){
    var m2 = 2 * m, aa = m * (b - m) * x / ((qam + m2) * (a + m2));
    d = 1 + aa * d; if (Math.abs(d) < FPMIN) d = FPMIN;
    c = 1 + aa / c; if (Math.abs(c) < FPMIN) c = FPMIN;
    d = 1 / d; h *= d * c;
    aa = -(a + m) * (qab + m) * x / ((a + m2) * (qap + m2));
    d = 1 + aa * d; if (Math.abs(d) < FPMIN) d = FPMIN;
    c = 1 + aa / c; if (Math.abs(c) < FPMIN) c = FPMIN;
    d = 1 / d; var del = d * c; h *= del;
    if (Math.abs(del - 1) < EPS) break;
  }
  return h;
}
function betainc(x, a, b){
  if (x <= 0) return 0; if (x >= 1) return 1;
  var bt = Math.exp(lgamma(a + b) - lgamma(a) - lgamma(b) + a * Math.log(x) + b * Math.log(1 - x));
  return x < (a + 1) / (a + b + 2) ? bt * betacf(a, b, x) / a : 1 - bt * betacf(b, a, 1 - x) / b;
}
function gammaP(a, x){
  if (x <= 0) return 0;
  if (x < a + 1){
    var ap = a, sum = 1 / a, del = sum;
    for (var n = 1; n <= 200; n++){ ap++; del *= x / ap; sum += del; if (Math.abs(del) < Math.abs(sum) * 1e-12)
break; }
    return sum * Math.exp(-x + a * Math.log(x) - lgamma(a));
  }
  var b = x + 1 - a, c = 1e300, d = 1 / b, h = d;
  for (var i = 1; i <= 200; i++){
    var an = -i * (i - a); b += 2; d = an * d + b; if (Math.abs(d) < 1e-300) d = 1e-300;
    c = b + an / c; if (Math.abs(c) < 1e-300) c = 1e-300;
    d = 1 / d; var delta = d * c; h *= delta;
    if (Math.abs(delta - 1) < 1e-12) break;
  }
  return 1 - Math.exp(-x + a * Math.log(x) - lgamma(a)) * h;
}
function erf(x){
  var s = x < 0 ? -1 : 1; x = Math.abs(x);
  var t = 1 / (1 + 0.3275911 * x);
  var y = 1 - (((((1.061405429 * t - 1.453152027) * t) + 1.421413741) * t - 0.284496736) * t + 0.254829592) * t
* Math.exp(-x * x);
  return s * y;
```

```
}
function normCdf(z){ return 0.5 * (1 + erf(z / Math.SQRT2)); }
function tP(t, df){ var x = df / (df + t * t); return betainc(x, df / 2, 0.5); } // two-sided p
function fP(f, d1, d2){ return betainc(d2 / (d2 + d1 * f), d2 / 2, d1 / 2); } // upper-tail p
function chiP(x, df){ return 1 - gammaP(df / 2, x / 2); } // upper-tail p
function tCrit(prob, df){ // quantile: P(T <= crit) = prob, via bisection on the two-sided p
  var target = 2 * (1 - prob); // two-sided p at the critical value
  var lo = 0, hi = 100;
  for (var i = 0; i < 80; i++){ var mid = (lo + hi) / 2; if (tP(mid, df) > target) lo = mid; else hi = mid; }
  return (lo + hi) / 2;
}
/* ---------- Formatting helpers ---------- */
function fmt(x, d){
  if (x === null || x === undefined || !isFinite(x)) return "–";
  var dec = d === undefined ? 3 : d;
  var v = Number(x);
  if (v !== 0 && (Math.abs(v) >= 1e6 || Math.abs(v) < 1e-4)) return v.toExponential(2);
  return v.toFixed(dec);
}
function fmtP(p){
  if (p === null || p === undefined || !isFinite(p)) return "–";
  if (p < 0.001) return "< .001";
  return p.toFixed(3).replace(/^0/, "");
}
function sigStars(p){ return p < 0.001 ? "***" : p < 0.01 ? "**" : p < 0.05 ? "*" : ""; }
/* ---------- Correlation ---------- */
function pearson(x, y){
  var n = x.length, mx = mean(x), my = mean(y), sxy = 0, sxx = 0, syy = 0;
  for (var i = 0; i < n; i++){ var dx = x[i] - mx, dy = y[i] - my; sxy += dx * dy; sxx += dx * dx; syy += dy *
dy; }
  if (sxx === 0 || syy === 0) return null;
  var r = sxy / Math.sqrt(sxx * syy);
  var t = r * Math.sqrt((n - 2) / (1 - r * r));
  return { r: r, n: n, t: t, p: n > 2 ? tP(t, n - 2) : null };
}
function ranks(a){
  var idx = a.map(function(v, i){ return [v, i]; }).sort(function(p, q){ return p[0] - q[0]; });
  var out = new Array(a.length), i = 0;
  while (i < idx.length){
    var j = i;
    while (j + 1 < idx.length && idx[j + 1][0] === idx[i][0]) j++;
    var avg = (i + j) / 2 + 1;
    for (var k = i; k <= j; k++) out[idx[k][1]] = avg;
    i = j + 1;
  }
  return out;
}
function spearman(x, y){ return pearson(ranks(x), ranks(y)); }

/* ---------- t-tests & ANOVA ---------- */
function tTestOne(a, mu0){
  var d = describe(a), t = (d.mean - mu0) / d.sem, df = d.n - 1;
  return { n: d.n, mean: d.mean, sd: d.sd, mu0: mu0, t: t, df: df, p: tP(t, df), diff: d.mean - mu0,
    ciLo: d.mean - mu0 - tCrit(0.975, df) * d.sem, ciHi: d.mean - mu0 + tCrit(0.975, df) * d.sem };
}
function tTestInd(a, b, pooled){
  var da = describe(a), db = describe(b), diff = da.mean - db.mean, se, df;
  if (pooled){
    var sp2 = ((da.n - 1) * da.variance + (db.n - 1) * db.variance) / (da.n + db.n - 2);
    se = Math.sqrt(sp2 * (1 / da.n + 1 / db.n)); df = da.n + db.n - 2;
  } else {
    var va = da.variance / da.n, vb = db.variance / db.n;
    se = Math.sqrt(va + vb); df = (va + vb) * (va + vb) / (va * va / (da.n - 1) + vb * vb / (db.n - 1));
  }
  var t = diff / se, tc = tCrit(0.975, Math.max(1, Math.round(df)));
  var dPool = Math.sqrt(((da.n - 1) * da.variance + (db.n - 1) * db.variance) / (da.n + db.n - 2));
  return { a: da, b: db, diff: diff, se: se, t: t, df: df, p: tP(t, df),
    ciLo: diff - tc * se, ciHi: diff + tc * se, cohensD: dPool > 0 ? diff / dPool : 0, pooled: pooled };
}
function tTestPaired(x, y){
  var diffs = x.map(function(v, i){ return v - y[i]; });
  var d = describe(diffs), t = d.mean / d.sem, df = d.n - 1;
  return { n: d.n, meanDiff: d.mean, sdDiff: d.sd, t: t, df: df, p: tP(t, df),
    ciLo: d.mean - tCrit(0.975, df) * d.sem, ciHi: d.mean + tCrit(0.975, df) * d.sem,
    cohensD: d.sd > 0 ? d.mean / d.sd : 0 };
}
function anovaOneWay(groups){ // groups: [{name, values}]
```

```
  var k = groups.length, all = [];
  groups.forEach(function(g){ all = all.concat(g.values); });
  var grand = mean(all), N = all.length, ssb = 0, ssw = 0;
  var stats = groups.map(function(g){
    var d = describe(g.values);
    ssb += g.values.length * (d.mean - grand) * (d.mean - grand);
    g.values.forEach(function(v){ ssw += (v - d.mean) * (v - d.mean); });
    return { name: g.name, d: d };
  });
  var dfB = k - 1, dfW = N - k, msb = ssb / dfB, msw = ssw / dfW, F = msb / msw;
  return { groups: stats, k: k, N: N, grand: grand, ssb: ssb, ssw: ssw, dfB: dfB, dfW: dfW,
    msb: msb, msw: msw, F: F, p: fP(F, dfB, dfW), eta2: ssb / (ssb + ssw) };
}

/* ---------- Chi-square ---------- */
function chiSquare(table){ // table: rows x cols of counts
  var R = table.length, C = table[0].length;
  var rowT = table.map(function(r){ return r.reduce(function(s, v){ return s + v; }, 0); });
  var colT = []; for (var j = 0; j < C; j++){ var s = 0; for (var i = 0; i < R; i++) s += table[i][j];
colT.push(s); }
  var N = rowT.reduce(function(s, v){ return s + v; }, 0);
  var chi2 = 0, lowCells = 0;
  var expected = table.map(function(r, i){ return r.map(function(o, j){
    var e = rowT[i] * colT[j] / N; if (e < 5) lowCells++;
    chi2 += (o - e) * (o - e) / e; return e;
  }); });
  var df = (R - 1) * (C - 1);
  return { chi2: chi2, df: df, p: chiP(chi2, df), N: N, expected: expected,
    cramersV: Math.sqrt(chi2 / (N * Math.min(R - 1, C - 1))), lowCellPct: 100 * lowCells / (R * C) };
}

/* ---------- Matrix helpers ---------- */
function matInverse(A){
  var n = A.length, M = A.map(function(r, i){ var row = r.slice(); for (var j = 0; j < n; j++) row.push(i ===
j ? 1 : 0); return row; });
  for (var col = 0, row = 0; col < n && row < n; col++, row++){
    var piv = row; for (var i = row + 1; i < n; i++) if (Math.abs(M[i][col]) > Math.abs(M[piv][col])) piv = i;
    if (Math.abs(M[piv][col]) < 1e-11) return null;
    var tmp = M[row]; M[row] = M[piv]; M[piv] = tmp;
    var div = M[row][col]; for (var j = 0; j < 2 * n; j++) M[row][j] /= div;
    for (var k = 0; k < n; k++){
      if (k === row) continue;
      var factor = M[k][col];
      if (factor !== 0) for (var j2 = 0; j2 < 2 * n; j2++) M[k][j2] -= factor * M[row][j2];
    }
  }
  return M.map(function(r){ return r.slice(n); });
}
function matMul(A, B){
  var n = A.length, m = B[0].length, p = B.length, out = [];
  for (var i = 0; i < n; i++){ out.push([]); for (var j = 0; j < m; j++){ var s = 0; for (var k = 0; k < p; k++)
s += A[i][k] * B[k][j]; out[i].push(s); } }
  return out;
}
/* OLS: y (array), X (array of predictor arrays), names (predictor names) */
function ols(y, X, names){
  var n = y.length, k = X.length + 1; // + intercept
  if (n <= k) return { error: "Not enough complete cases (" + n + ") for " + (k - 1) + " predictor(s). Remove
missing data or predictors." };
  var XtX = [], XtY = [];
  for (var i = 0; i < k; i++){ XtX.push(new Array(k).fill(0)); XtY.push(0); }
  function xv(row, j){ return j === 0 ? 1 : X[j - 1][row]; }
  for (var r = 0; r < n; r++) for (var a = 0; a < k; a++){
    XtY[a] += xv(r, a) * y[r];
    for (var b = 0; b < k; b++) XtX[a][b] += xv(r, a) * xv(r, b);
  }
  var inv = matInverse(XtX);
  if (!inv) return { error: "The predictors are perfectly (or nearly perfectly) collinear — the model cannot be
estimated. Remove a redundant predictor." };
  var beta = inv.map(function(r2){ return r2.reduce(function(s, v, j){ return s + v * XtY[j]; }, 0); });
  var ybar = mean(y), rss = 0, tss = 0;
  for (var r3 = 0; r3 < n; r3++){
    var fit = 0; for (var j2 = 0; j2 < k; j2++) fit += beta[j2] * xv(r3, j2);
    rss += (y[r3] - fit) * (y[r3] - fit); tss += (y[r3] - ybar) * (y[r3] - ybar);
  }
  var dfRes = n - k, s2 = rss / dfRes, r2 = tss > 0 ? 1 - rss / tss : 0;
  var coef = beta.map(function(b, j){
```

```
    var se = Math.sqrt(Math.max(0, s2 * inv[j][j])), t = se > 0 ? b / se : 0;
    return { name: j === 0 ? "(Constant)" : names[j - 1], b: b, se: se, t: t, p: tP(t, dfRes),
      betaStd: 0 };
  });
  // standardised coefficients
  var sy = Math.sqrt(variance(y, true));
  for (var j3 = 1; j3 < k; j3++){
    var sx = Math.sqrt(variance(X[j3 - 1], true));
    coef[j3].betaStd = sy > 0 ? beta[j3] * sx / sy : 0;
  }
  var F = dfRes > 0 && rss > 0 ? ((tss - rss) / (k - 1)) / (rss / dfRes) : 0;
  return { coef: coef, r2: r2, adjR2: 1 - (1 - r2) * (n - 1) / dfRes, F: F, fp: fP(F, k - 1, dfRes),
    dfModel: k - 1, dfRes: dfRes, sigma: Math.sqrt(s2), n: n };
}
/* ---------- Eigendecomposition (Jacobi, symmetric matrices) ---------- */
function jacobiEigen(Ain){
  var n = Ain.length, A = Ain.map(function(r){ return r.slice(); });
  var V = []; for (var i = 0; i < n; i++){ V.push([]); for (var j = 0; j < n; j++) V[i].push(i === j ? 1 : 0); }
  for (var sweep = 0; sweep < 100; sweep++){
    var off = 0; for (var p = 0; p < n; p++) for (var q = p + 1; q < n; q++) off += A[p][q] * A[p][q];
    if (off < 1e-14) break;
    for (var p2 = 0; p2 < n - 1; p2++) for (var q2 = p2 + 1; q2 < n; q2++){
      if (Math.abs(A[p2][q2]) < 1e-15) continue;
      var theta = 0.5 * Math.atan2(2 * A[p2][q2], A[q2][q2] - A[p2][p2]);
      var c = Math.cos(theta), s = Math.sin(theta);
      for (var k = 0; k < n; k++){
        var akp = A[k][p2], akq = A[k][q2];
        A[k][p2] = c * akp - s * akq; A[k][q2] = s * akp + c * akq;
      }
      for (var k2 = 0; k2 < n; k2++){
        var apk = A[p2][k2], aqk = A[q2][k2];
        A[p2][k2] = c * apk - s * aqk; A[q2][k2] = s * apk + c * aqk;
      }
      for (var k3 = 0; k3 < n; k3++){
        var vkp = V[k3][p2], vkq = V[k3][q2];
        V[k3][p2] = c * vkp - s * vkq; V[k3][q2] = s * vkp + c * vkq;
      }
    }
  }
  var vals = []; for (var d = 0; d < n; d++) vals.push(A[d][d]);
  var order = vals.map(function(v, i){ return i; }).sort(function(a, b){ return vals[b] - vals[a]; });
  return { values: order.map(function(i){ return vals[i]; }),
    vectors: order.map(function(i){ var col = []; for (var r = 0; r < n; r++) col.push(V[r][i]); return
col; }) };
}
function corrMatrix(colsData){ // colsData: array of complete-case columns
  var p = colsData.length, R = [];
  for (var i = 0; i < p; i++){ R.push([]); for (var j = 0; j < p; j++){
    var r = pearson(colsData[i], colsData[j]); R[i].push(r ? r.r : 0);
  } }
  return R;
}
/* PCA on standardised data (correlation-based) */
function pca(colsData, names){
  var R = corrMatrix(colsData), eig = jacobiEigen(R), p = colsData.length;
  var total = eig.values.reduce(function(s, v){ return s + Math.max(0, v); }, 0);
  var comps = eig.values.map(function(ev, ci){
    var loadings = eig.vectors[ci].map(function(v){ return v * Math.sqrt(Math.max(0, ev)); });
    return { eigenvalue: ev, pct: total > 0 ? 100 * Math.max(0, ev) / total : 0, loadings: loadings };
  });
  var cum = 0; comps.forEach(function(c){ cum += c.pct; c.cumPct = cum; });
  return { components: comps, names: names.slice(), totalVar: total };
}
/* Varimax rotation (loadings: rows = variables, cols = factors) */
function varimax(L, iters){
  var p = L.length, f = L[0].length;
  if (f < 2) return L.map(function(r){ return r.slice(); });
  var R = L.map(function(r){ return r.slice(); });
  for (var it = 0; it < (iters || 100); it++){
    var maxAng = 0;
    for (var i = 0; i < f - 1; i++) for (var j = i + 1; j < f; j++){
      // standard varimax angle (Kaiser): u = a^2-b^2, v = 2ab aggregates
      var su = 0, sv = 0, suv = 0, suu = 0, svv = 0;
      for (var v2 = 0; v2 < p; v2++){
        var a2 = R[v2][i], b2 = R[v2][j], uu = a2 * a2 - b2 * b2, vv = 2 * a2 * b2;
        su += uu; sv += vv; suv += uu * vv; suu += uu * uu; svv += vv * vv;
      }
```

```
      var numA = 2 * suv - 2 * su * sv / p;
      var denA = suu - svv - (su * su - sv * sv) / p;
      var ang = 0.25 * Math.atan2(numA, denA);
      maxAng = Math.max(maxAng, Math.abs(ang));
      if (Math.abs(ang) > 1e-10){
        var c = Math.cos(ang), s = Math.sin(ang);
        for (var v3 = 0; v3 < p; v3++){
          var x = R[v3][i], y = R[v3][j];
          R[v3][i] = c * x - s * y; R[v3][j] = s * x + c * y;
        }
      }
    }
    if (maxAng < 1e-6) break;
  }
  return R;
}
/* Factor analysis: principal axis factoring with iterated communalities + varimax */
function factorAnalysis(colsData, names, nf){
  var p = colsData.length, R = corrMatrix(colsData);
  var inv = matInverse(R.map(function(r){ return r.slice(); }));
  var h2 = [];
  for (var i = 0; i < p; i++) h2.push(inv && inv[i][i] > 0 ? Math.min(0.99, Math.max(0.05, 1 - 1 / inv[i][i])) :
0.5);
  var L = null;
  for (var iter = 0; iter < 100; iter++){
    var Rr = R.map(function(r, ri){ return r.map(function(v, ci){ return ri === ci ? h2[ri] : v; }); });
    var eig = jacobiEigen(Rr);
    L = [];
    for (var v = 0; v < p; v++){
      L.push([]);
      for (var f = 0; f < nf; f++){
        var ev = Math.max(0, eig.values[f] || 0);
        L[v].push((eig.vectors[f] ? eig.vectors[f][v] : 0) * Math.sqrt(ev));
      }
    }
    var newH2 = L.map(function(row){ return row.reduce(function(s, x){ return s + x * x; }, 0); });
    var change = 0; for (var v2 = 0; v2 < p; v2++) change = Math.max(change, Math.abs(newH2[v2] - h2[v2]));
    h2 = newH2.map(function(x){ return Math.min(0.999, x); });
    if (change < 1e-4) break;
  }
  var rot = varimax(L);
  // order factors by explained variance (desc) and orient signs so largest loading is positive
  var ss = [];
  for (var f2 = 0; f2 < nf; f2++){ var s = 0; for (var v3 = 0; v3 < p; v3++) s += rot[v3][f2] * rot[v3][f2];
ss.push(s); }
  var order = ss.map(function(v, i){ return i; }).sort(function(a, b){ return ss[b] - ss[a]; });
  var loadings = rot.map(function(row){
    return order.map(function(f){ return row[f]; });
  });
  loadings.forEach(function(row, v4){
    // nothing per-row; sign orientation per factor below
  });
  for (var f3 = 0; f3 < nf; f3++){
    var maxRow = 0, maxAbs = 0;
    loadings.forEach(function(row, v5){ if (Math.abs(row[f3]) > maxAbs){ maxAbs = Math.abs(row[f3]); maxRow =
v5; } });
    if (loadings[maxRow][f3] < 0) loadings.forEach(function(row){ row[f3] = -row[f3]; });
  }
  var ssOrd = order.map(function(f){ return ss[f]; });
  return { loadings: loadings, communalities: h2, names: names.slice(),
    ssLoadings: ssOrd, pct: ssOrd.map(function(s){ return 100 * s / p; }) };
}
```